

CMAA5043 Creative Prototyping

Lab 4 (Cursor Lab 1) Tutorial (Part I): Building a 2D Game with AI-Assisted Development

Objectives: By the end of this lab, you will be able to:

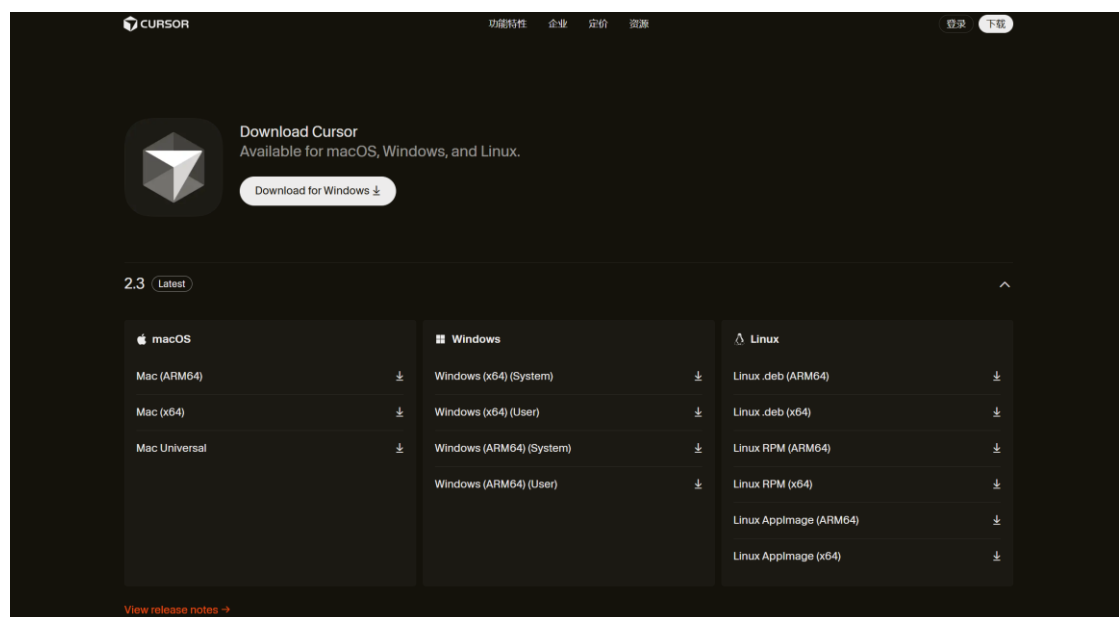
1. Set up and navigate the Cursor IDE.
2. Apply structured prompting to generate functional code.
3. Build and debug a simple interactive prototype using HTML5 Canvas.
4. Iterate on a prototype by describing new features.

Final Deliverable: Cursor Setup, a working, keyboard-controlled 2D game in a single HTML file.

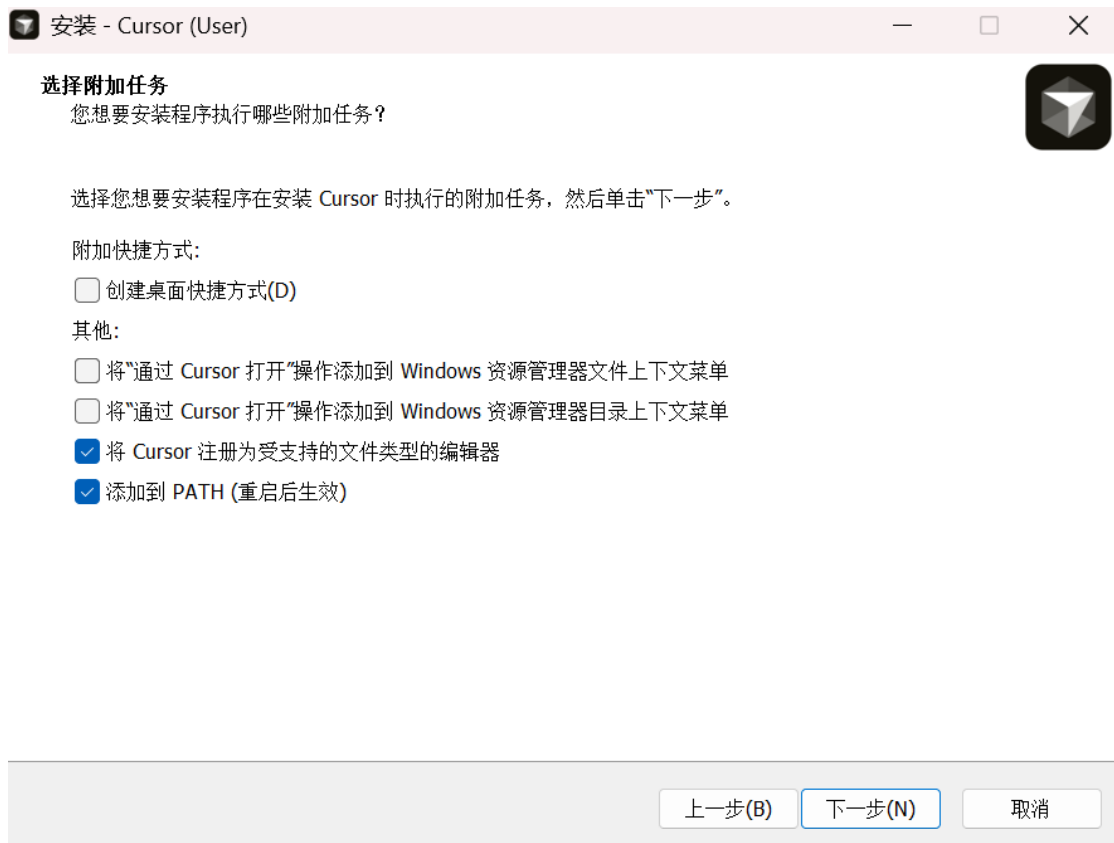
Task 0: Installation and Setup

Step 0.1: Install Cursor

1. Go to the official [Cursor website](https://cursor.sh).



2. Download the version for your operating system (macOS, Windows, or Linux).
3. Install the application. On first launch, you may need to grant necessary system permissions for file access.

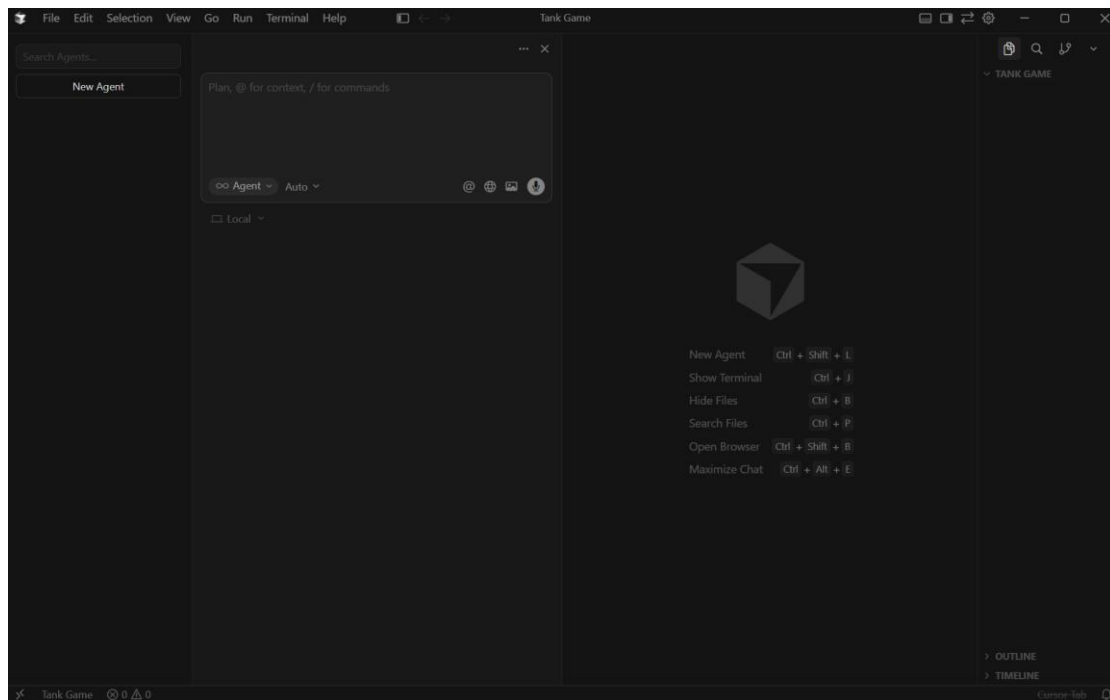


Step 0.2: Configure Your Workspace

1. Create a new folder on your computer named Tank Game.

You can choose to connect to a folder, a github repository, or a remote server folder via SSH.

2. Open Cursor, and use File > Open... to open this folder. Your workspace should look like the figure below.

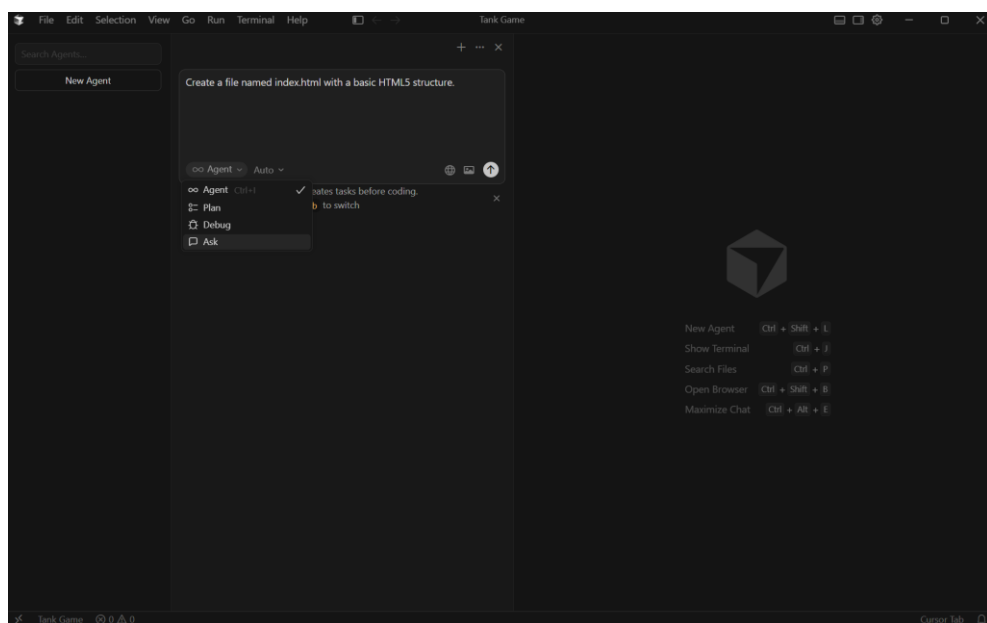


Task 1: Your First Prompt – Creating the Project Structure

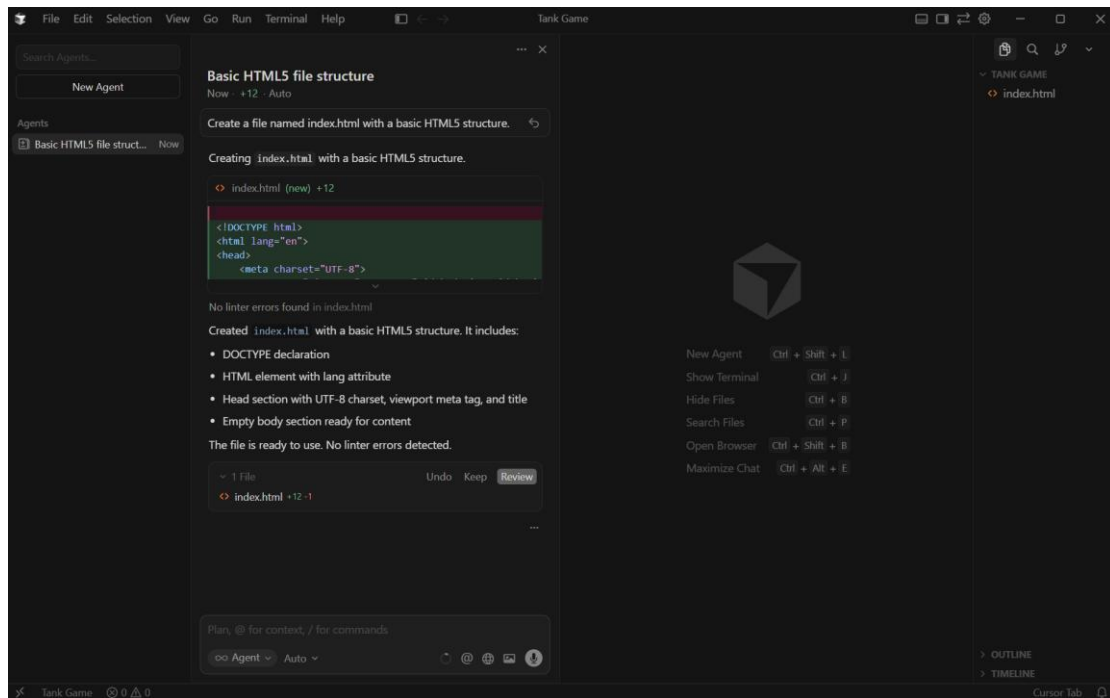
In this task, we will learn the most basic yet crucial skill: giving Cursor clear, contextual instructions.

Step 1.1: Create the Base File

1. You can note that there are different mode of cursor agent. Let's choose agent and type in the chat panel:



“Create a file named index.html with a basic HTML5 structure.”



2. Cursor will create the file and generate the boilerplate code. You can choose to keep, undo, or review the generated code. After clicking review, you can see a the new script on the right side of the screen with tracked changes.

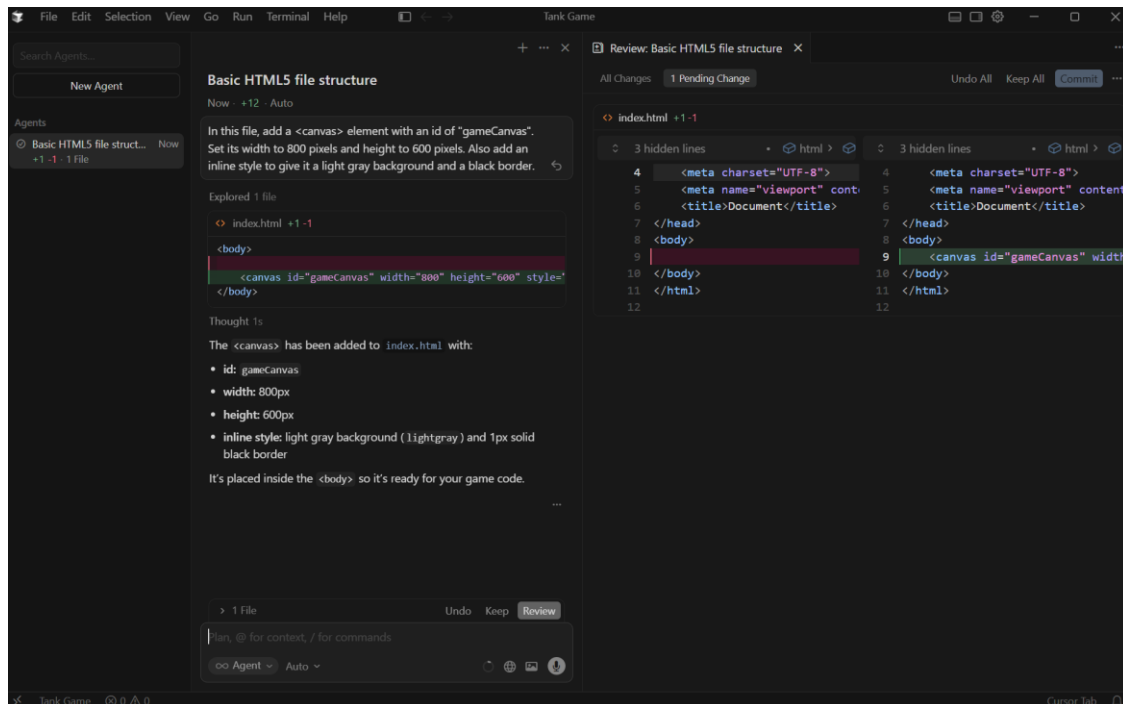
Step 1.2: Set Up the Canvas

We need a drawing area for our game. Update the prompt with more specific context.

1. Ensure index.html is open in the editor. In the Chat panel, type:

In this file, add a `<canvas>` element with an id of "gameCanvas". Set its width to 800 pixels and height to 600 pixels. Also add an inline style to give it a light gray background and a black border.

2. Observe how Cursor modifies the existing file, adding the `<canvas>` element inside the `<body>` tag. Check the code and keep the changes.



Task 2: Structured Prompting – Drawing the Tank

Complex tasks must be broken down. We will build the tank by describing its components step-by-step.

Step 2.1: Create the JavaScript Logic

1. First, ask Cursor to add a `<script>` tag for our game logic.

“Right before the closing `</body>` tag, add a `<script>` tag. Inside it, write JavaScript to get a reference to the canvas element and its 2D drawing context.”

2. Now, let's define the tank. We'll describe it as a function that takes a position.

“Following the code you just wrote, create a function named `drawTank(ctx, x, y)`. It should draw:

1. A green rectangle (40x20 pixels) for the tank body, centered around (x, y).
2. A smaller dark green rectangle (30x10 pixels) for the turret, centered on top of the body.
3. A thin black line (20 pixels long) extending from the turret to represent the gun barrel.

3. Finally, call the function to see our tank on the screen.

After the `drawTank` function, write code to call `drawTank` once, positioning it at

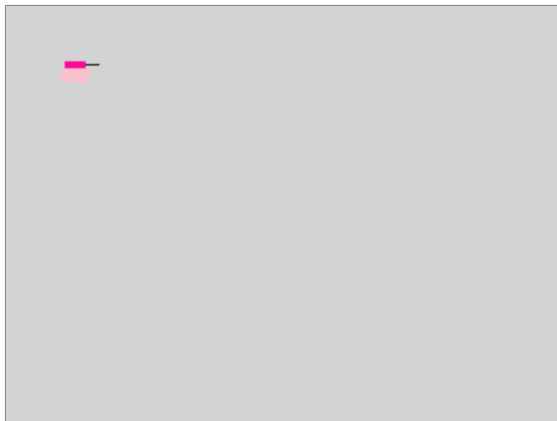
coordinates (100, 100). "

Step 2.2: Test Your Prototype

1. Save the index.html file.



2. Open it directly in your web browser (e.g., by double-clicking the file). You should see a static tank drawn on the canvas, as shown in the figure below.
3. You can adjust the appearance of the tank by prompting the Canvas, e.g., "the thin black line should be attached to the right of the dark green rectangle not on the rectangle" or "change the color of the tank to pink."



Key Concept: We used **structured prompting** by breaking the tank into sub-components (body, turret, barrel) and specifying their **relative position and size**. This gives Cursor a clear blueprint to generate accurate code.

Task 3: Making it Interactive – Prompting for Game Logic

The prototype of a tank game should be interactive. We will now prompt the game's dynamic behavior.

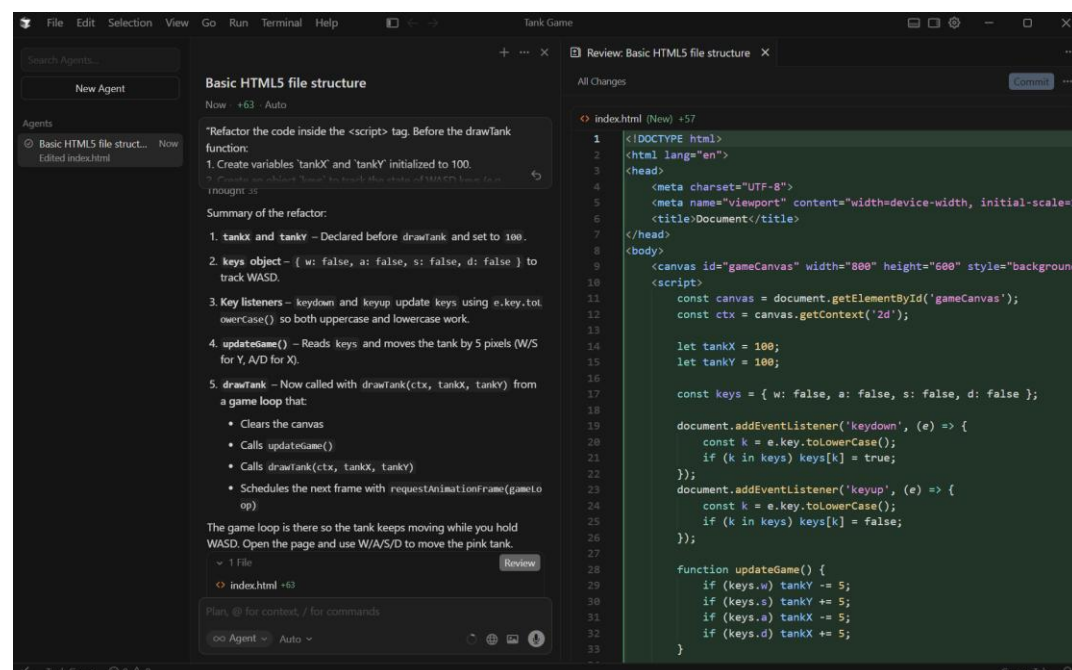
Step 3.1: Add Keyboard Controls

We need variables to store the tank's state and event listeners to update it.

1. Ask Cursor to set up the game state and loop. A good prompt references existing code.

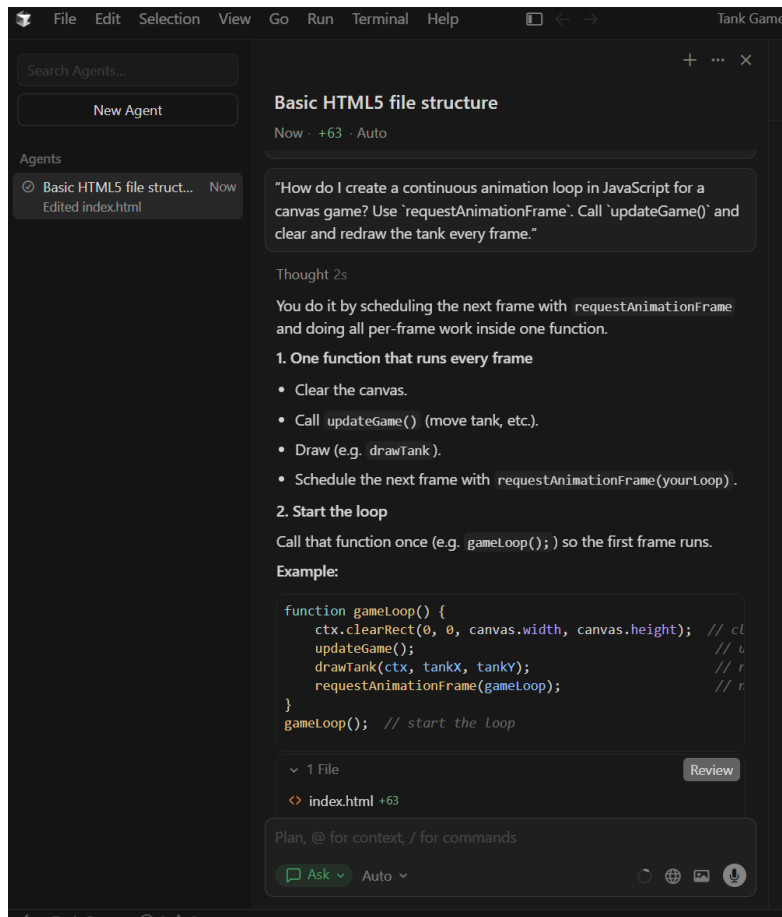
“Refactor the code inside the `<script>` tag. Before the `drawTank` function:

1. Create variables `tankX` and `tankY` initialized to 100.
2. Create an object `keys` to track the state of WASD keys (e.g., `{w: false, a: false, s: false, d: false}`).
3. Set up `keydown` and `keyup` event listeners to update the `keys` object.
4. Create a function `updateGame()` that checks the `keys` object and updates `tankX` or `tankY` by 5 pixels per press.
5. Modify the call to `drawTank` to use `tankX` and `tankY`.”



2. Now, we need to make the game animate. Try asking Cursor using a Ask chatbot instead of an Agentic chatbot.

“How do I add instructions in JavaScript for a this tank game?”



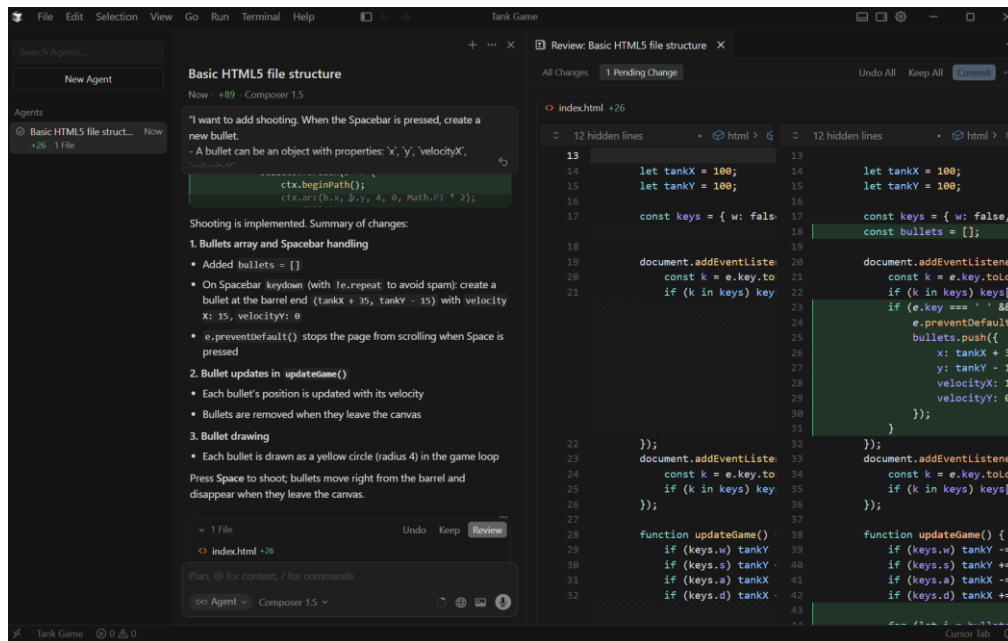
Step 3.2: Implementing Shooting Mechanics

Let's add the core action.

1. First, describe how bullets should be represented and managed.

"I want to add shooting. When the Spacebar is pressed, create a new bullet.

- A bullet can be an object with properties: `x`, `y`, `velocityX`, `velocityY`.
- Store all active bullets in an array `bullets`.
- In `updateGame()`, update each bullet's position and remove it if it goes off-screen.
- In the drawing loop, draw each bullet as a small yellow circle."



Debugging Tip: If something doesn't work, **copy the browser's error message or describe the unexpected behavior** and paste it into the Cursor Chat.

Cursor Lab 1 (Part II) Tutorial: GitHub Integration – Version Control for Creative Prototypes

Objectives:

By the end of this lab, you will be able to:

1. Create a GitHub repository and connect it to your local project.
2. Use Cursor's built-in Git tools to stage, commit, and push code.
3. Understand the basics of version control in a creative prototyping workflow.
4. Use Cursor to write commit messages and manage branches.

Final Deliverable:

A GitHub repository containing your tank game from Part I, with at least 3 commits and a proper .gitignore file.

Task 0: Prerequisites

Before starting this lab, ensure you have:

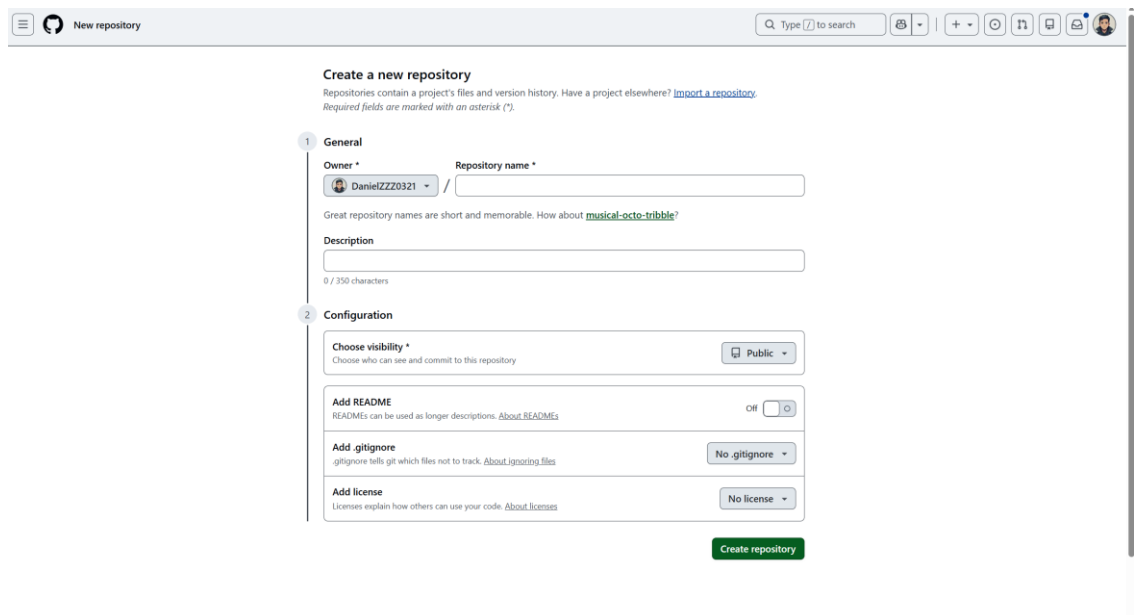
1. A GitHub account (sign up at github.com if you don't have one).
2. Git installed on your computer (<https://git-scm.com/downloads>).
3. Cursor IDE installed and configured (completed in Part I).
4. Your Tank Game project folder from Part I.

Task 1: Setting Up a GitHub Repository

In this task, we will create a remote repository on GitHub and connect it to your local project.

Step 1.1: Create a New Repository on GitHub

1. Go to [GitHub.com](https://github.com) and log in.
2. Click the "+" icon in the top-right corner and select "New repository".



The screenshot shows the GitHub 'Create a new repository' page. The page has a header with the GitHub logo, a search bar, and navigation icons. The main content area is titled 'Create a new repository' and includes a sub-header 'Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository.](#)' and a note 'Required fields are marked with an asterisk (*)'.

The form is divided into two sections: 'General' and 'Configuration'.

General

- Owner ***: A dropdown menu showing 'DanielZZ0321'.
- Repository name ***: A text input field.
- Description**: A text input field with a character count '0 / 350 characters'.

Configuration

- Choose visibility ***: A dropdown menu showing 'Public'.
- Add README**: A section with a toggle switch set to 'off' and a link 'About READMEs'.
- Add .gitignore**: A section with a dropdown menu showing 'No .gitignore' and a link 'About ignoring files'.
- Add license**: A section with a dropdown menu showing 'No license' and a link 'About licenses'.

A green 'Create repository' button is located at the bottom right of the form.

3. Fill in the repository details:
 - Repository name: tank-game-prototype
 - Description (optional): "A simple 2D tank game built with Cursor AI"
 - Visibility: Public (or Private, depending on your preference)
 - Do NOT initialize with a README, .gitignore, or license (we already have

a project).

4. Click "Create repository".

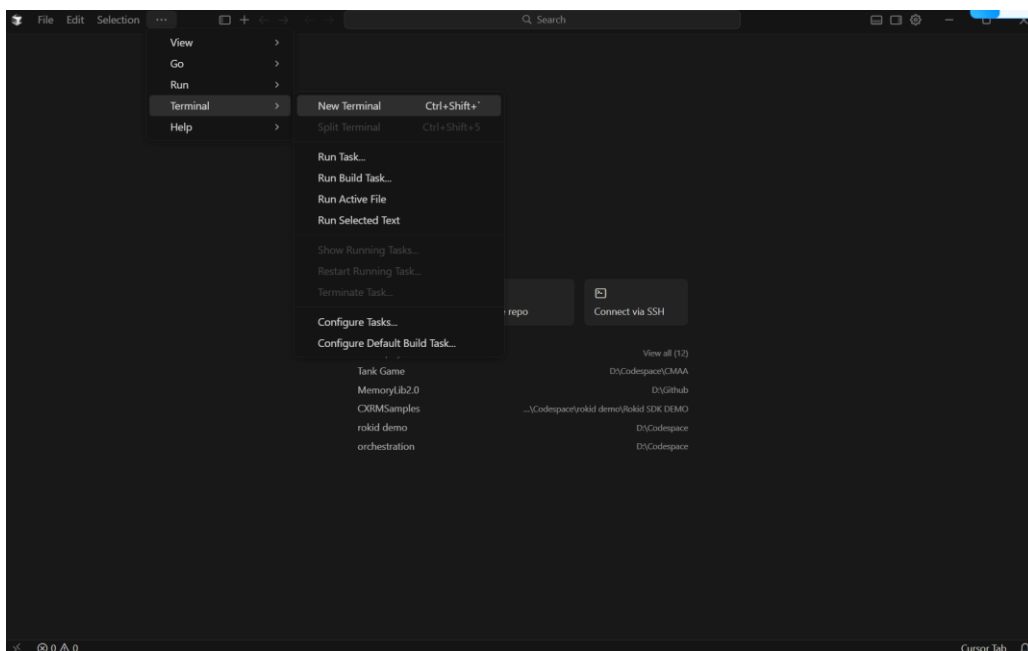
You will now see a page with instructions for connecting an existing repository. Keep this page open — we'll use the commands shown there.

Task 2: Connecting Your Local Project to GitHub

Now we will link your local Tank Game folder to the GitHub repository you just created.

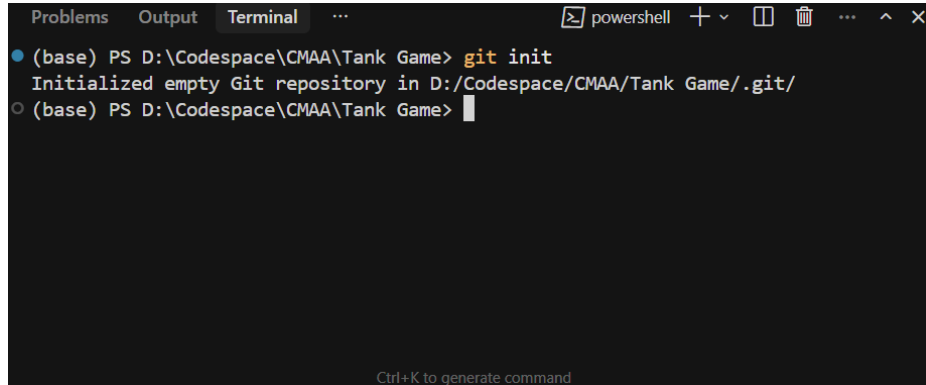
Step 2.1: Initialize Git in Your Local Folder

1. Open Cursor and ensure your Tank Game folder is open (File > Open...).
2. Open the Terminal in Cursor: Terminal > New Terminal (or use the shortcut Ctrl+`).



3. In the terminal, type the following commands one by one:

git init



```
(base) PS D:\Codespace\CMAA\Tank Game> git init
Initialized empty Git repository in D:/Codespace/CMAA/Tank Game/.git/
(base) PS D:\Codespace\CMAA\Tank Game>
```

This initializes a new Git repository in your current folder.

Step 2.2: Add and Commit Your Files

Now we need to track all your game files and make the first commit.

1. Stage all files (add them to Git's tracking system):

```
git add .
```

2. Commit the files with a meaningful message:

```
git commit -m "Initial commit: tank game with basic movement and shooting"
```

You should see output indicating that files like index.html have been committed.

Step 2.3: Link to GitHub Repository

Now we connect your local repo to the remote one on GitHub.

1. Go back to the GitHub page from Step 1.1. Copy the remote URL under "...or push an existing repository from the command line". It will look like:

<https://github.com/yourusername/tank-game-prototype.git>

2. In the Cursor terminal, run:

```
git remote add origin https://github.com/yourusername/tank-game-prototype.git
```

3. Push your code to GitHub:

```
git branch -M main
```

```
git push -u origin main
```

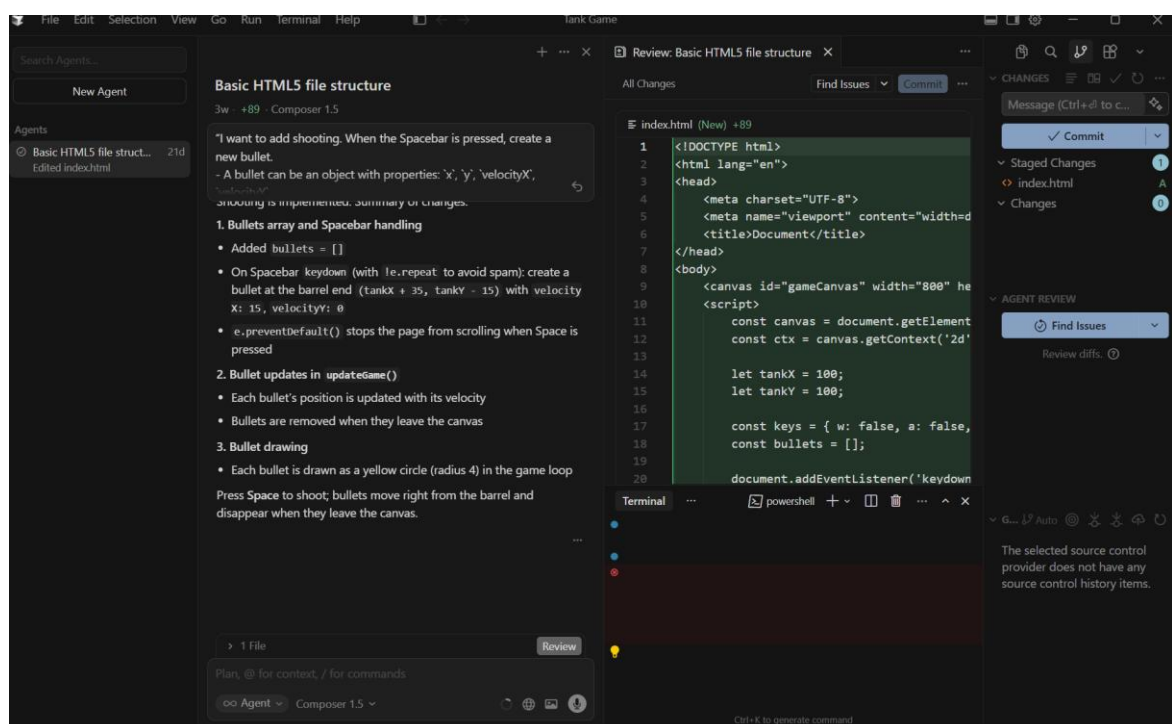
4. Refresh your GitHub repository page. You should now see all your files (including index.html) online

Task 3: Using Cursor's Built-in Git Tools

Cursor integrates Git functionality directly into the IDE. This makes it easy to stage, commit, and push without using the terminal.

Step 3.1: View Changes in the Source Control Panel

1. In Cursor, click on the Source Control icon in the left sidebar (it looks like a small branch, or use Ctrl+Shift+G).
2. You'll see a list of files that have been changed (modified) or are untracked (new).
3. Hover over a file and click the "+" icon to stage it, or click the "-" to discard changes.



Step 3.2: Make a Change and Commit via Cursor

Let's modify the game and commit the change using the GUI.

1. Open index.html and make a small change — for example, change the tank's color to blue.
 - Find the drawTank function and modify the fillStyle for the body.
2. Save the file (Ctrl+S).
3. Go to the Source Control panel.
4. You'll see index.html listed under Changes.

5. Click the "+" next to the file to stage it.
6. In the Message box at the top, type: "changed tank color to blue"
7. Click the checkmark (✓) to commit.

Step 3.3: Push Changes to GitHub

After committing locally, you need to push to GitHub.

1. In the Source Control panel, click the "..." (more actions) menu.
2. Select "Push".
3. Your changes are now live on GitHub!

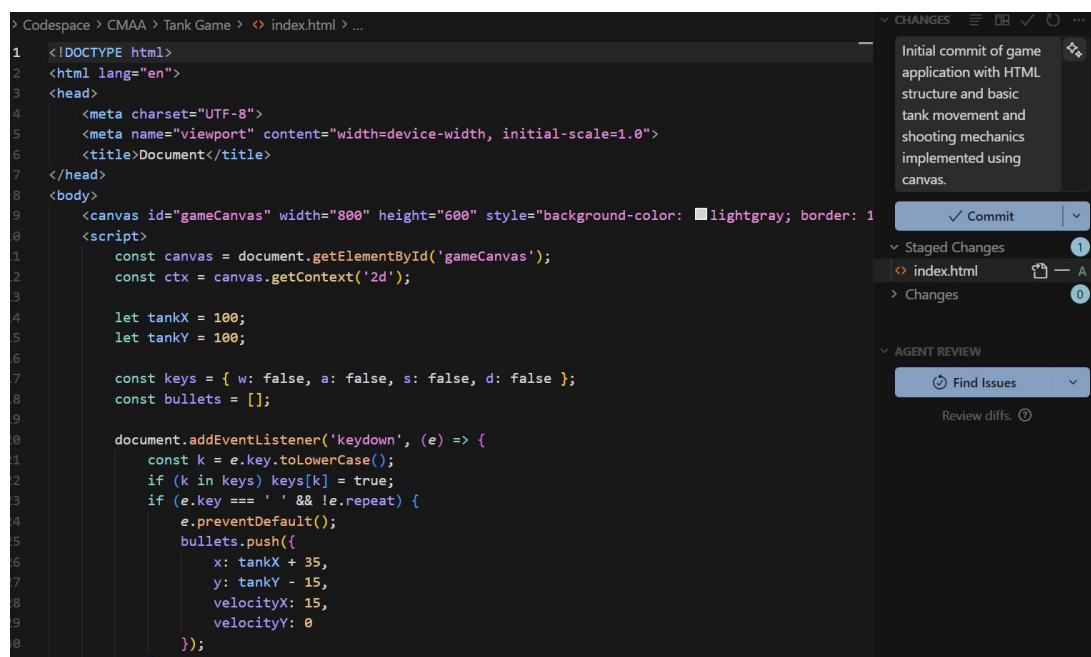
Task 4: Advanced Git Workflows with Cursor AI

One of Cursor's strengths is its ability to help you write commit messages and even suggest branching strategies.

Step 4.1: AI-Generated Commit Messages

Cursor can write meaningful commit messages for you based on your changes.

1. Stage your changes (as in Step 3.2).
2. In the Source Control panel, click the sparkle icon (✨) next to the message box.



3. Cursor will analyze your changes and suggest a commit message like:
"Update tank color to blue and adjust barrel position"
4. You can accept, edit, or regenerate the message.
5. Commit and push.

Step 4.2: Create a New Branch for Experimentation

Branches allow you to experiment without breaking the main game.

1. In the terminal, create a new branch:

```
git checkout -b experimental-turret
```

2. Make a change — for example, make the turret rotate with the mouse.
3. Stage and commit this change using the Source Control panel.
4. Push the new branch to GitHub:

```
git push -u origin experimental-turret
```

5. Go to GitHub and you'll see your new branch listed.

Task 5: Best Practices for Creative Prototyping with Git

To wrap up, let's ensure your repository is clean and professional.

Step 5.1: Add a .gitignore File

Some files (like OS metadata or temporary files) should not be in Git.

1. Ask Cursor to generate a .gitignore file for you:

"Create a .gitignore file for a simple HTML/JavaScript project, ignoring common OS and editor files."

2. Cursor will generate something like:

OS files

```
.DS_Store  
Thumbs.db
```

Editor files

```
.vscode/  
*.swp
```

*.SWO

Logs

*.log

3. Save the file as .gitignore in your project root.
4. Stage and commit it.

Step 5.2: Write a README.md

A good README explains what your project is and how to run it.

1. Create a new file called README.md.
2. Ask Cursor to help:

"Write a README.md for a simple 2D tank game. Include: title, description, how to run, controls (WASD to move, Space to shoot), and credits."

3. Adjust as needed, then stage and commit.